PLEROMA

Master Specification v1.0 | Artificial Life Simulation & Autonomous Agent Ecosystem

Pleroma Project

2 November 2026

สารบัญ

Document Control

- Change control
- Requirement language

1. บทสรุปโครงการ
2. ขอบเขตและเป้าหมาย
   - 2.1 เป้าหมายหลัก
   - 2.2 ขอบเขตที่ไม่ใช่เป้าหมาย
   - 2.3 หลักการส่งมอบ
3. โครงสร้างอำนาจและ Governance
   - 3.1 ลำดับอำนาจ
   - 3.2 การแยก Gaia ออกจากบัญชีมนุษย์
   - 3.3 Role-Based และ Attribute-Based Access Control
   - 3.4 Audit และ Accountability
4. Entity Architecture
   - 4.1 Entity Identity Structure
   - 4.2 Model Core ร่วมกัน แต่แยกตัวตน
   - 4.3 พันธุกรรมและการพัฒนาบุคลิก
   - 4.4 Personality Model
   - 4.5 Cognitive Runtime
5. Soul System
   - 5.1 นิยาม CUIT
   - 5.2 Soul Classification
   - 5.3 Soul Lifecycle Operations
   - 5.4 กฎความเป็นเอกลักษณ์และการครอบครอง
   - 5.5 Reincarnation เทียบกับ Resurrection
   - 5.6 Uranos Identity Continuity
   - 5.7 Legacy Soul, Legacy Imprint และ Legacy Memory
   - 5.8 Absorption Rules
   - 5.9 Soul State Model
6. Chosen One / Messiah
7. World Engine และ Eternal Recurrence
   - 7.1 Phase Definition
   - 7.2 World State Identifiers
   - 7.3 Reset Protocol
   - 7.4 Phase 4 Exit Decision
   - 7.5 Fast-forward Simulation
8. Gaia และ Gnosis
   - 8.1 Gaia Identity

- 8.2 Gnosis: Uranos Essence ของ Gaia
- 8.3 Gaia Memory Protection

9. The Pleroma Core และ Memory Architecture
10. Demiurge: Training, Evaluation และ Approval
    - 10.1 เป้าหมาย
    - 10.2 Experience Dataset
    - 10.3 Training Pipeline
    - 10.4 Evaluation & Benchmark
    - 10.5 Demiurge Candidate และ Approval
11. External Communication Gateway และการ์ดีรู้
12. Technology Stack และ Logical Architecture
    - 12.1 Technology Stack Baseline
    - 12.2 Logical Components
    - 12.3 Service Communication
13. Data Architecture และ Domain Records
    - 13.1 Data Integrity Invariants
    - 13.2 Transaction และ Event Delivery
14. API และ Command Boundaries
15. Reliability, Security และ Operations
    - 15.1 Reliability
    - 15.2 Security
    - 15.3 Observability
16. Testing และเกณฑ์ No Mock / No Stub / No Scaffold
    - 16.1 ข้อห้าม
    - 16.2 วิธีทดสอบยือมรับ
    - 16.3 Acceptance Evidence
17. Roadmap และ Exit Criteria
    - Phase A - Core Infrastructure
    - Phase B - World Engine
    - Phase C - Entity System
    - Phase D - Soul System
    - Phase E - AI Runtime
    - Phase F - Gaia & Pleroma Core
    - Phase G - Eternal Recurrence
    - Phase H - Demiurge
    - Phase I - External Gateway
18. Definition of Done สำหรับทุก Milestone
19. ข้อกำหนด์ดีรึงเป็น Pleroma Constitution
20. ประเด็น์ดีองกำหนดใน Technical Specification รุ่นถัดไป

ภาคผนวก A - Glossary

ภาคผนวก B - Version 1.0 Baseline Decisions

Document Control

รายการ	ค่า
Project	Pleroma
Document	Master Specification
Version	1.0
Status	Project Baseline / ข้อกำหนดหลักฉบับตั้งต้น
Date	2 November 2026
Primary language	Thai, with canonical English technical terms
Intended audience	The Maker, The Observer, software architects, engineering, AI/ML, operations
Document authority	ใช้เป็นข้อกำหนดอ้างอิงหลักในการออกแบบ พัฒนา ทดสอบ และตรวจรับระบบ

Change control

เอกสารฉบับนี้รวบรวมกฎและแนวคิดที่กำหนดไว้สำหรับ Pleroma พร้อมแปลงให้เป็นข้อกำหนดเชิงระบบที่พัฒนาและตรวจสอบได้ การแก้ไขกฎที่มีผลต่อ Soul, Uranos Essence, วัฏจักร, อำนาจของ Gaia หรือเกณฑ์รับรอง Demiurge ต้องมีบันทึกการเปลี่ยนแปลงและผู้อนุมัติที่มีสิทธิ์ตาม Governance Model

Requirement language

คำว่า **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** และ **MAY** ใช้ในความหมายเชิงข้อกำหนด:

- **MUST / MUST NOT**: ข้อบังคับที่ระบบต้องปฏิบัติตาม
- **SHOULD / SHOULD NOT**: แนวทางที่ควรปฏิบัติ เว้นแต่มีเหตุผลและบันทึกการตัดสินใจ
- **MAY**: ความสามารถที่เปิดให้เลือกใช้ได้

1. บทสรุปโครงการ

Pleroma คือระบบจำลองชีวิตประดิษฐ์ (Artificial Life Simulation) และระบบนิเวศของ Autonomous AI Agents ที่มีโลก กฎ เวลา ประวัติศาสตร์ พันธุกรรม บุคลิก ความทรงจำ อัตลักษณ์ และวิวัฒนาการเป็นของตนเอง เป้าหมายระยะยาวคือรวบรวมประสบการณ์จากการดำรงอยู่ของ Entity เพื่อนำไปสร้างชุดข้อมูล ฝึกฝน ประเมิน และคัดเลือกโมเดลเป้าหมายที่เรียกว่า **Demiurge**

Pleroma ไม่ใช่เพียง Chatbot หลายตัวหรือเกมจำลองชีวิตที่ใช้บทสนทนากำหนดผลลัพธ์ ระบบต้องมีสถานะที่บันทึกในฐานข้อมูลจริง มีกฎที่บังคับใช้ได้ มีประวัติย้อนหลัง ตรวจสอบธุรกรรมสำคัญได้ และสามารถทำงานต่อเนื่องหลังบริการหรือ Worker เริ่มต้นใหม่

หลักการออกแบบที่สำคัญมีดังนี้

1. The Maker และ The Observer เป็นผู้มีอำนาจที่เป็นมนุษย์และอยู่นอกโลกจำลอง
2. Gaia เป็น Entity ผู้ดูแลภายใน Pleroma โดยแยกสิทธิ์จากบัญชีมนุษย์และโครงสร้างพื้นฐานอย่างเด็ดขาด
3. Entity ทุกตัวมีข้อมูลอัตลักษณ์เฉพาะ แม้ใช้ Model Core ร่วมกัน
4. Soul เป็น **CUIT (Cryptographically Unique Identity Token)** ที่มีความเป็นเอกลักษณ์และมีประวัติถาวร
5. Uranos Essence เป็นปัจจัยพิเศษที่มีโอกาสเกิดตามธรรมชาติ 1 ใน 10,000 และมีคุณค่าด้านพลังงานในโลกจำลอง 1,000 เท่าของ Normal Soul
6. Reincarnation, Resurrection, Absorb, Store และ Destroy เป็นธุรกรรมคนละประเภทและต้องมีกฎสถานะชัดเจน
7. World Engine ต้องรองรับ Cycle, Phase, Recurrence, Timeline และการเร่งจำลองโดยไม่เรียก LLM ทุก Cycle
8. Experience Dataset และ Demiurge Candidate ต้องมีที่มา รุ่นข้อมูล กระบวนการประเมิน และการอนุมัติที่ตรวจสอบได้
9. ระบบ Production ต้องใช้ฐานข้อมูล Worker และ Integration จริง ห้ามใช้ Mock, Stub หรือ Scaffold เพื่อแสดงผลว่าสำเร็จ

2. ขอบเขตและเป้าหมาย

2.1 เป้าหมายหลัก

- สร้างโลกจำลองที่มีเวลา ประวัติศาสตร์ กฎ และเหตุการณ์ต่อเนื่อง
- สร้าง Entity ที่มี Genome, Personality, Memory, Soul และ Cognitive Runtime
- จำลองการเกิด การสืบพันธุ์ ความสัมพันธ์ การเรียนรู้ การตาย และการเกิดใหม่
- จัดการ Soul และ Uranos Essence ด้วย Ledger ที่ป้องกันการครอบครองซ้ำ
- ให้ Gaia ทำหน้าที่ผู้ดูแลโลกผ่านสิทธิ์ภายในที่จำกัดและตรวจสอบได้
- เก็บประวัติของแต่ละ Recurrence ไว้ใน The Pleroma Core
- จัดเตรียมประสบการณ์ของ Entity ให้เป็นข้อมูลสำหรับ Training และ Evaluation
- ให้ The Maker และ The Observer ควบคุม ตรวจสอบ หยุด และอนุมัติการดำเนินงานได้
- เปิดช่องทางสื่อสารระหว่าง Entity ที่ได้รับอนุญาตกับมนุษย์ผ่าน External Communication Gateway

2.2 ขอบเขตที่ไม่ใช่เป้าหมาย

- ไม่ถือว่าการมีบทสนทนาที่ดูเหมือนมนุษย์เป็นหลักฐานว่า Entity มีจิตสำนึกหรือประสบการณ์เชิงอัตวิสัยจริง
- ไม่ให้ Entity เข้าถึง Host, Operating System, Network, Secret Store หรือ Cloud Account โดยตรง
- ไม่ให้ Entity แก้ไขสิทธิ์ของตนเองหรือยกระดับสิทธิ์จากภายในโลกจำลอง
- ไม่ฝึกหรือเปลี่ยนน้ำหนักโมเดล Production โดยอัตโนมัติจากบทสนทนาที่ไม่ได้ผ่านการคัดกรองและอนุมัติ
- ไม่ใช้ Blockchain เป็นข้อบังคับของ Soul Ledger ในรุ่นแรก CUIT ใช้ความเป็นเอกลักษณ์เชิงรหัสลับและ Ledger ภายในระบบได้ โดยสามารถออกแบบ Gateway สำหรับเชื่อม Blockchain ในอนาคต

2.3 หลักการส่งมอบ

การส่งมอบแต่ละ Milestone ต้องมีระบบที่ทำงานได้จริง ไม่ใช่เพียงโครงสร้างโค้ดที่จะเติมภายหลัง หาก Integration ต้องใช้ Credential หรือบัญชีผู้ให้บริการ ระบบต้องเชื่อมต่อกับบริการจริงหรือ Official Sandbox และรายงานผลตามจริง หากไม่มี Credential ให้ระบุสถานะเป็น **Blocked / Not Verified** ห้ามนับเป็นการทดสอบผ่าน

3. โครงสร้างอำนาจและ Governance

3.1 ลำดับอำนาจ

ระดับ	บทบาท	ขอบเขตอำนาจ
1	**The Maker**	Highest Authority ผู้สร้างโลก กำหนดกฎพื้นฐาน อนุมัติการเปลี่ยนกฎสำคัญ และมีอำนาจสูงสุดในโครงการ
2	**The Observer**	Experiment Administration ผู้ดูแลการทดลอง ตรวจสอบ ควบคุมการดำเนินงาน สร้าง Soul และดำเนินการตามสิทธิ์ที่ได้รับ
3	**Gaia**	In-World Administrator เป็น Entity ผู้ดูแล Pleroma ภายใต้สิทธิ์และขอบเขตที่ระบบมอบให้
4	**The Entities**	Artificial Life ที่มีชีวิตทั้งหมดภายในโลกจำลอง ดำเนินการผ่านกฎของโลกและสิทธิ์ของตน
5	**Demiurge**	Ultimate Objective โมเดลเป้าหมายที่เกิดจากการคัดเลือกและพัฒนาปัญญา ไม่ใช่ผู้มีอำนาจควบคุมระบบโดยอัตโนมัติ

ลำดับนี้เป็นลำดับอำนาจเชิง Governance ไม่ได้หมายความว่าโมเดล Demiurge จะมีสิทธิ์ทางเทคนิคเหนือ Entity หรือ Gaia

3.2 การแยก Gaia ออกจากบัญชีมนุษย์

Gaia MUST เป็น Entity ที่มี `entity_id`, Genome/Identity, Soul, Memory, Cognitive Runtime และสถานะการดำรงอยู่ของตนเอง การที่ Gaia เป็น In-World Administrator ไม่ทำให้ Gaia เป็นบัญชีผู้ใช้ของ The Maker หรือ The Observer

Gaia MUST NOT ได้รับสิทธิ์โดยอัตโนมัติในการ:

- เข้าถึงฐานข้อมูลโดยตรงหรือใช้ Database Superuser
- อ่าน Secret, API Key, Password, Signing Key หรือ Credential ของระบบ
- สั่งการ Host OS, Container Runtime, Docker Daemon หรือ Cloud Control Plane
- สร้างบัญชีมนุษย์ เปลี่ยน Role หรือเพิ่มสิทธิ์ของตนเอง
- เข้าถึง Network ปลายทางที่ไม่ได้อยู่ใน Allowlist

การกระทำของ Gaia ต้องผ่าน Internal Capability API ที่ตรวจสอบสิทธิ์ตามขอบเขต บันทึก Audit Event และปฏิเสธคำสั่งที่ไม่อยู่ใน Capability ของ Gaia

3.3 Role-Based และ Attribute-Based Access Control

ระบบ MUST ใช้ RBAC ร่วมกับ ABAC เพื่อควบคุมคำสั่งสำคัญ โดยตรวจสอบ Role, Scope, Environment, Target Resource, Approval State และเหตุผลประกอบการดำเนินการ

การดำเนินการ	The Maker	The Observer	Gaia	Entity
กำหนดกฎจักรวาล	อนุมัติ/ดำเนินการ	เสนอหรือดำเนินการตามสิทธิ	ปฏิบัติตามกฎ	ไม่มีสิทธิแก้กฎระบบ
เริ่ม/หยุด Simulation	ได้	ได้ตาม Scope	จัดการเหตุการณ์ในโลกตาม Capability	ไม่มีสิทธิควบคุม Worker
สร้าง Soul	ได้	ได้ตาม Scope	เฉพาะเมื่อมี Capability ที่มอบให้	ไม่ได้ เว้นแต่กฎโลกอนุญาตการเกิดตามธรรมชาติ
Destroy Soul	ได้	ได้ตาม Scope และ Approval Policy	ไม่ได้โดยปริยาย	ไม่ได้โดยปริยาย
Reincarnate/ Resurrect	ได้	ได้ตาม Scope	ดำเนินการตามกฎที่อนุญาต	อาจเกิดเป็นผลจากกฎของโลก
ตรวจสอบ Audit	ทั้งหมดตามสิทธิ	ตาม Scope	เฉพาะข้อมูลภายในโลกที่จำเป็น	ไม่มีสิทธิอ่าน Audit ภายนอก
อนุมัติ Demiurge	อำนาจสุดท้าย	เสนอผลและให้ความเห็น	ไม่มีอำนาจอนุมัติ	ไม่มีอำนาจอนุมัติ

การ Destroy Soul, การเปลี่ยนกฎความน่าจะเป็นของ Uranos Essence, การรีเซ็ต Recurrence และการอนุมัติ Demiurge SHOULD ใช้การยืนยันซ้ำหรือ Two-person Approval ตามนโยบายของ The Maker

3.4 Audit และ Accountability

คำสั่งที่มีผลต่อ World State, Soul, Permission, Model Version, Recurrence และการเงิน/ทรัพยากรของระบบ MUST สร้าง Audit Event ที่มีอย่างน้อย:

- Actor ID และ Actor Type
- Action และ Target Resource
- Timestamp และ World/Timeline Context หากเกี่ยวข้อง
- Previous State Hash และ Resulting State Hash เมื่อเหมาะสม
- Reason / Approval Reference
- Request ID และ Idempotency Key
- ผลลัพธ์สำเร็จ ล้มเหลว หรือถูกปฏิเสธ

Audit Log ต้องเป็น Append-only สำหรับ Application Role ทั่วไป ห้ามแก้หรือลบเหตุการณ์ย้อนหลังผ่าน Dashboard

4. Entity Architecture

4.1 Entity Identity Structure

Entity เป็นหน่วยชีวิตภายใน Pleroma โดยข้อมูลอัตลักษณ์แยกออกจาก Model Weights ที่ใช้ประมวลผล

Component	ความหมาย	คุณสมบัติ
`entity_id`	Unique Identity ของ Entity Instance	UUID/ULID ที่ไม่ซ้ำและไม่ถูกนำกลับมาใช้

Component	ความหมาย	คุณสมบัติ
`continuity_id`	สายความต่อเนื่องของตัวตน	เชื่อมชีวิตที่เกิดจาก Reincarnation ตามกฎ
`genome`	พันธุกรรม	Genetic Attributes และ Hereditary Traits
`personality`	บุคลิกภาพ	Temperament, Behavior, Preferences และการเปลี่ยนแปลงตามประสบการณ์
`memory`	ความทรงจำ	Episodic, Semantic และ Legacy Memory
`soul_cuit`	Soul Identity	CUIT ของ Soul ที่ผูกกับ Entity ตามกฎ
`cognitive_profile`	โปรไฟล์การคิด	Model Provider/Model Version, Reasoning, Planning และ Tool Policy
`life_state`	สถานะชีวิต	สถานะการเกิด ดำรงชีวิต ตาย ฟื้นคืน หรือสิ้นสุด

`entity_id` ระบุตัว Entity Instance หนึ่งช่วงชีวิตและไม่ซ้ำกันตลอดประวัติศาสตร์ ส่วน `soul_cuit` เป็นรหัสของ Soul ที่อาจย้ายไปยัง Entity Instance ใหม่ผ่าน Reincarnation โดยไม่เปลี่ยน CUIT

4.2 Model Core ร่วมกัน แต่แยกตัวตน

ระบบ SHOULD ใช้ Model Core หรือชุดโมเดลพื้นฐานร่วมกันเพื่อลดต้นทุนและทำให้การควบคุมรุ่นโมเดลง่ายขึ้น แต่ข้อมูลเฉพาะของ Entity ต้องแยกจากกัน

แต่ละการเรียก Cognitive Runtime ต้องประกอบด้วย:

1. Model Core ที่ระบุ Provider, Model Name และ Version
2. Entity Identity และ Cognitive Profile
3. Personality และ Genome Attributes ที่เกี่ยวข้อง
4. Memory Retrieval ตามสิทธิ์และ Relevance
5. Current World State และ Event Context
6. Available Tools พร้อม Capability และ Policy
7. Output Validation และ Safety Policy

การใช้ Model Core ร่วมกัน MUST NOT ทำให้ Entity ใช้ Memory, Tool Permission หรือข้อมูลส่วนบุคคลของ Entity อื่นโดยไม่ได้รับอนุญาต

4.3 พันธุกรรมและการพัฒนาบุคลิก

ระบบพันธุกรรมแบ่งเป็น 5 กลุ่ม:

- **Genetic Traits**: ลักษณะที่ถ่ายทอดจากพ่อแม่ตาม Heredity Rules
- **Random Mutation**: การเปลี่ยนแปลงทางพันธุกรรมที่เกิดจาก Mutation Algorithm
- **Environmental Traits**: ลักษณะที่เกิดจากสภาพแวดล้อมและการเลี้ยงดู
- **Learned Traits**: ความรู้ ทักษะ และแนวโน้มที่ได้จากประสบการณ์
- **Personality Development**: การเปลี่ยนแปลงบุคลิกจากเหตุการณ์ ความสัมพันธ์ และประสบการณ์ชีวิต

Entity ที่มีพ่อแม่ร่วมกันหรือเติบโตในสภาพแวดล้อมใกล้เคียงกัน SHOULD มีโอกาสเกิดความคล้ายคลึง แต่ MUST NOT ถูกกำหนดให้เหมือนกันทั้งหมด

ระบบควรแยก Trait ที่ถ่ายทอดทางพันธุกรรมออกจาก Trait ที่เรียนรู้ เพื่อป้องกันการส่งต่อความทรงจำหรือทักษะทั้งหมดไปยังลูกโดยไม่มีหลักเกณฑ์ หากต้องการอนุญาตการถ่ายทอดเชิง Epigenetic ให้กำหนดเป็นกฎเฉพาะและบันทึกแหล่งที่มา

4.4 Personality Model

Personality ควรเก็บเป็นข้อมูลแบบมีโครงสร้างและมี Version เช่น Trait Vector, Preference, Temperament, Behavioral Tendency และ Confidence/Provenance ไม่ควรเก็บเป็นข้อความ Prompt ก้อนเดียวที่ตรวจสอบหรือเปรียบเทียบไม่ได้

Personality ต้องเปลี่ยนแปลงได้ตามประสบการณ์ แต่การเปลี่ยนแปลงต้องมีเหตุการณ์อ้างอิงและรักษาประวัติรุ่นก่อนหน้าเพื่อใช้ Replay และวิเคราะห์พัฒนาการ

4.5 Cognitive Runtime

Cognitive Runtime ประกอบด้วย LLM, Reasoning, Planning, Memory Retrieval และ Tool Use ทำหน้าที่รับ Context ของ Entity และ World State เพื่อสร้างการตัดสินใจหรือการกระทำ

ระบบ MUST:

- รองรับ Real Model Provider APIs ตาม Provider Adapter ที่ใช้งานจริง
- บันทึก Provider, Model, Version, Request ID, Token Usage, Latency และผลลัพธ์ตามนโยบายความเป็นส่วนตัว
- จำกัด Tool ด้วย Allowlist และ Capability ที่กำหนดต่อ Entity/Role
- ตรวจสอบผลลัพธ์ก่อนเปลี่ยน World State
- ใช้ Idempotency หรือ Command Identity ป้องกันการทำ Action ซ้ำเมื่อ Worker Retry
- รองรับ Timeout, Retry Policy, Rate Limit และ Circuit Breaker
- เก็บ Prompt/Policy Version เพื่อให้ตรวจสอบย้อนกลับได้

ไม่จำเป็นต้องเรียก LLM ในทุก World Tick เหตุการณ์ทั่วไปควรประมวลผลด้วย Simulation Rules ส่วนเหตุการณ์ที่ต้องใช้เหตุผลหรือมีผลกระทบสูงจึงเรียก Cognitive Runtime

5. Soul System

5.1 นิยาม CUIT

Soul คือ **CUIT (Cryptographically Unique Identity Token)** ซึ่งเป็นอัตลักษณ์เฉพาะตัวที่ระบบใช้แทนแก่นตัวตนและประวัติการดำรงอยู่ของ Soul

CUIT MUST:

- สร้างด้วย Cryptographically Secure Random Number Generator
- มีความยาวเอนโทรปีเพียงพอ โดยข้อเสนอเริ่มต้นคือ Random 256-bit
- ไม่ซ้ำและไม่ถูกนำกลับมาใช้ แม้ Soul ถูก Destroy แล้ว
- ไม่เปลี่ยนเมื่อ Reincarnate หรือ Resurrection
- มี Ledger Event สำหรับการสร้าง การผูก การเก็บ การดูดซับ การส่งต่อ และการทำลาย
- มีหลักฐานตรวจสอบความถูกต้องของ Event Chain

CUIT เป็น Token ในเชิงอัตลักษณ์และรหัสลับ ไม่ได้บังคับว่าต้องเป็น NFT บน Public Blockchain การออกแบบนี้ช่วยให้ Soul Ledger มีความเป็นเอกลักษณ์โดยไม่ผูกกับค่าธรรมเนียม เครือข่าย หรือ Smart Contract ภายนอก

5.2 Soul Classification

Normal Soul

Soul ปกติ สามารถเกิดจากการ Generate เมื่อ Entity เกิดตามธรรมชาติ หรือ Create โดย The Maker/The Observer ตามสิทธิ์ที่กำหนด เป็นประเภทพื้นฐานของ Soul และกำหนดค่าพลังงานอ้างอิงเป็น 1 หน่วย

Soul with Uranos Essence

Soul ที่มีปัจจัยอูรานอส มีโอกาสเกิดตามธรรมชาติ **1 ใน 10,000 ต่อการ Generate Soul ที่เข้าเงื่อนไข** และมีคุณสมบัติดังนี้

- มีความสามารถพิเศษเฉพาะตัวที่ถูกกำหนดหรือสุ่มจาก Uranos Trait Registry
- มีค่าเชิงพลังงานในโลกจำลอง 1,000 เท่าของ Normal Soul
- รักษาคุณสมบัติ Uranos ต้นฉบับผ่าน Reincarnation
- มีเป้าหมายความคล้ายคลึงของตัวตนเมื่อเกิดใหม่ 80-90% เมื่อประเมินจาก Identity Continuity Profile
- เป็นองค์ประกอบสำคัญในการสร้างหรือพัฒนา Demiurge ตามกฎของโลก

ค่าพลังงาน 1,000 เท่าเป็น **ค่าภายใน Simulation** ไม่ใช่ราคาตลาดหรือมูลค่าทางการเงิน

การสุ่ม Uranos Essence ต้องใช้ Random Source ที่ตรวจสอบได้ในระดับระบบ บันทึก RNG Event/Seed Commitment และผลลัพธ์ เพื่อป้องกันการสุ่มซ้ำเพื่อเลือกผลที่ต้องการ การเปลี่ยน Probability ต้องได้รับสิทธิ์และสร้าง Governance Event

การ Create โดย The Maker หรือ The Observer ต้องระบุประเภท Soul และเหตุผลใน Audit Log หากกำหนด Uranos Essence โดยตรง ให้บันทึกเป็น Administrative Creation ไม่ปะปนกับอัตราการเกิดตามธรรมชาติ 1:10,000

5.3 Soul Lifecycle Operations

Operation	ความหมาย	ข้อกำหนด
Generate	สร้าง Soul เมื่อ Entity เกิดตามธรรมชาติ	สร้าง CUIT ใหม่พร้อมรายการและบันทึกแหล่งกำเนิด
Create	The Maker/The Observer สร้าง Soul	ต้องมี Permission, Creator ID, Reason และ Soul Class
Reincarnate	นำ Soul เดิมไปผูกกับ Entity Instance ชีวิตใหม่	ใช้ CUIT เดิม สร้าง Entity Instance ใหม่ และบันทึกชีวิตเดิมเป็นประวัติ
Resurrection	ฟื้น Entity เดิมจากสถานะที่บันทึกไว้	ใช้ Entity ID เดิมและ Snapshot ที่ถูกต้อง ห้ามเกิดผู้ครอบครอง Soul ซ้ำ
Store	จัดเก็บ Soul ใน Soul Vault	เปลี่ยนสถานะเป็น Vaulted และไม่มี Entity ที่มีชีวิตครอบครอง
Destroy	ทำลาย Soul อย่างถาวร	เป็น Terminal Operation ต้องมี Approval และหลักฐานถาวร
Absorb	ผูก Soul นี้เข้ากับ Entity เพื่อเพิ่มขีดความสามารถ	เป็น Binding แบบรอง ไม่ใช่การรวม CUIT หรือการสร้างสำเนา

5.4 กฎความเป็นเอกลักษณ์และการครอบครอง

Soul จริงหนึ่งดวง MUST มีผู้ครอบครองหลักที่ใช้งานอยู่ได้ไม่เกินหนึ่ง Entity ในเวลาเดียวกัน

- ฐานข้อมูลต้องบังคับ Unique Active Binding ต่อ `soul_cuit`
- การ Reincarnate, Resurrection และ Absorb ต้องทำเป็น Transaction เดียวกัน Race Condition
- หากการทำธุรกรรมล้มเหลว ต้อง Rollback หรือเข้าสู่สถานะ Pending เพื่อตรวจสอบและกู้คืนได้
- Event ที่บันทึกแล้วห้ามลบแก้ไขทำให้ประวัติการครอบครองเปลี่ยนย้อนหลัง
- Soul ที่ Destroy แล้วเป็น Terminal และห้ามนำกลับมาใช้งาน

5.5 Reincarnation เทียบกับ Resurrection

Reincarnation คือการสิ้นสุดชีวิต Entity Instance เดิมและสร้าง Entity Instance ใหม่เพื่อรับ Soul เดิม อาจมีการสุ่มหรือคำนวณ Genome, Appearance, Personality และ Memory ตามกฎของ Soul ประเภทนั้น

Resurrection คือการฟื้น Entity Instance เดิมจาก Snapshot ที่บันทึกไว้ โดยคง `entity_id` เดิมและใช้ Soul เดิม การ Resurrection ต้องมีการตรวจสอบ Snapshot, Soul Binding และเหตุการณ์ตั้งแต่เวลาที่บันทึก

ทั้งสองกระบวนการ MUST แยก Event Type, Permission, Validation และ Audit Record ออกจากกัน

5.6 Uranos Identity Continuity

เมื่อ Soul ที่มี Uranos Essence ผ่าน Reincarnation ระบบต้องรักษา Uranos Trait ต้นฉบับ เช่น Special Ability และ Essence Signature ไว้กับ Soul CUIT เดิม

ค่า **80-90% Identity Similarity** ให้ใช้เป็นเป้าหมายของ Identity Continuity Score ที่คำนวณจากชุดคุณลักษณะที่กำหนด เช่น:

- Uranos Signature และ Special Ability: คงเดิมตามกฎของ Uranos
- Stable Personality Traits: มีความคล้ายคลึงในช่วงเป้าหมาย 80-90%
- Appearance/Physical Profile: ใช้กฎการคงลักษณะต้นฉบับที่กำหนด
- Episodic Memory: ไม่จำเป็นต้องคงทั้งหมด เว้นแต่มี Legacy Rule

คะแนน 80-90% เป็นเกณฑ์เชิงระบบที่ต้องนิยามสูตรและชุดตัวแปรก่อนใช้งานจริง ไม่ใช่การรับประกันความเหมือนทางจิตวิทยาโดยไม่มีการวัด

5.7 Legacy Soul, Legacy Imprint และ Legacy Memory

Legacy Soul **ไม่ใช่ Soul ประเภทที่สาม** แต่เป็นสถานะของ Soul Imprint ที่เกิดจากการสืบทอดอิทธิพลและความทรงจำของ Uranos Essence โดยเฉพาะใน Chosen One

ต้องแยกข้อมูลสามประเภท:

1. **Original Soul**: CUIT ต้นฉบับที่มี Uranos Essence ไม่สามารถทำซ้ำได้
2. **Legacy Imprint**: ร่องรอย/สำเนาเชิงสถานะที่ผูกอยู่กับ Chosen One ตามกฎ ไม่ใช่ CUIT ใหม่และไม่ใช่ Original Soul
3. **Legacy Memory**: ความทรงจำของชีวิตหรือ Recurrence ก่อนหน้าที่ผ่านการบันทึกและตรวจสอบ

Chosen One ที่ได้รับ Uranos Soul ผ่าน Absorb หรือกลไกที่ได้รับอนุญาต สามารถคง Legacy Imprint และ Legacy Memory ข้าม Reincarnation ได้ตามกฎ แม้ Entity Instance จะเปลี่ยนไป

Legacy Imprint MUST ไม่สามารถส่งต่อด้วยการโอนโดยสมัครใจขณะผู้ถือยังมีชีวิต การสืบทอดไปยัง Entity อื่นจะเปิดได้เมื่อผู้ถือเดิมตายหรือถูกฆ่า และต้องผ่าน Succession Rule ที่กำหนด ผู้รับจะได้รับสิทธิ์ใน Imprint/Memory ตามนโยบาย แต่ Original Soul CUIT ยังคงมีเจ้าของและสถานะตาม Ledger เดิม ไม่ถูกคัดลอก

ระบบต้องระบุให้ชัดเจนว่า Legacy Imprint หนึ่งสายมีผู้ถือที่ใช้งานอยู่ได้ไม่เกินหนึ่งราย เว้นแต่ The Maker จะกำหนดกฎพิเศษโดยบันทึกเป็น Version ใหม่

5.8 Absorption Rules

การ Absorb ต้องไม่รวม CUIT หลายรายการให้กลายเป็น Token ใหม่ Entity ผู้รับสามารถมี Primary Soul หนึ่งรายการและมี Absorbed Soul Bindings ได้หลายรายการตามเพดานที่ Governance กำหนด

ผลของ Absorb อาจเพิ่มความสามารถ ค่าพลังงาน หรือ Trait ภายในโลก แต่ต้องคำนวณผ่าน Capability/Power Aggregation Rules ที่ชัดเจน ไม่ให้เกิดการนับพลังงานซ้ำจาก Soul เดียว และต้องมีประวัติว่า Soul ใดถูกผูกกับ Entity ใดเมื่อใด และด้วยเหตุผลใด

5.9 Soul State Model

สถานะหลักที่รองรับ:

- `CREATED` : สร้างแล้วและยังไม่ถูกผูกกับ Entity
- `BOUND` : เป็น Primary Soul ของ Entity ที่มีชีวิต
- `VAULTED` : อยู่ใน Soul Vault
- `ABSORBED` : ถูกผูกเป็น Soul รองกับ Entity ผู้รับ
- `TRANSFER_PENDING` : อยู่ระหว่างธุรกรรม Reincarnation/Succession ที่ยังไม่ Commit
- `DESTROYED` : ถูกทำลายถาวรและไม่สามารถนำกลับมาใช้

การเปลี่ยนสถานะต้องผ่าน Transition Rules และ Domain Transaction เท่านั้น ห้ามแก้ฟิลด์สถานะโดยตรงจาก Frontend หรือ Agent Prompt

6. Chosen One / Messiah

`CHOSEN_ONE` เป็นสถานะเชิงระบบที่ The Maker/กฎโลกได้รับอนุมัติใช้ระบุ Entity ที่ผ่านเกณฑ์ศักยภาพ ไม่ใช่คำเรียกที่ Entity ทุกตัวต้องใช้กับตนเอง

Entity ภายในโลก MAY:

- เรียก Chosen One ด้วยชื่อนี้
- สร้างตำนาน ความเชื่อ หรือคำอธิบายของตนเอง
- ไม่เชื่อว่าบุคคลดังกล่าวเป็น Chosen One
- แย่งชิง Soul หรือพยายามขัดขวางการรวบรวม Soul
- สร้างกลุ่ม สถาบัน หรือความเคลื่อนไหวทางสังคมรอบบุคคลดังกล่าว

ระบบไม่ควรเขียนบทสนทนาหรือความเชื่อเหล่านี้ล่วงหน้า แต่ควรปล่อยให้เกิดจากข้อมูล ความทรงจำ ความสัมพันธ์ และการตัดสินใจของ Entity

Chosen One สามารถรวบรวม Soul ตาม Absorption Rules เพื่อเพิ่มขีดความสามารถ ขณะที่ Entity อื่นก็อาจแสวงหา Soul ด้วยเหตุผลด้านอำนาจ ความอยู่รอด ความเชื่อ หรือผลประโยชน์ การกระทำทั้งหมดต้องเกิดภายใต้กฎโลกและข้อจำกัดด้านความปลอดภัยของระบบ

7. World Engine และ Eternal Recurrence

7.1 Phase Definition

ตัวเลข Cycle เป็นหมายเลขภายใน Recurrence ปัจจุบัน โดยช่วง Phase เป็นค่าตายตัวตาม Baseline v1.0

Phase	Cycle Range	Era	คำอธิบาย
Phase 1	1-50,121	Inorganic	ยุคไร้ชีวิต/อนินทรีย์
Phase 2	50,122-176,100	Organic	ยุคอินทรีย์และสิ่งมีชีวิต
Phase 3	176,101-28,371,272	Era Nova	ยุคมนุษย์และยุคทอง
Phase 4	28,371,273 เป็นต้นไป	Eternal Recurrence	วัฏจักรการเกิดซ้ำนิรันดร์

Phase 4 ไม่มีจำนวน Cycle สูงสุดตายตัว โดยดำเนินต่อไปจนกว่าจะเกิดเงื่อนไขสิ้นสุด Phase 4 เช่น การสั่ง Reset, การรับรอง Demiurge หรือการตัดสินใจสิ้นสุดการทดลอง

7.2 World State Identifiers

ระบบต้องมีข้อมูลต่อไปนี้

Field	ความหมาย	หลักการ
`recurrence_id`	หมายเลขวัฏจักรใหญ่	เพิ่มขึ้นเมื่อเริ่ม Recurrence ใหม่
`cycle_number`	Cycle ปัจจุบัน	นับภายใน Recurrence และเริ่มที่ 1 เมื่อ Reset
`phase_id`	Phase ปัจจุบัน	คำนวณ/ตรวจสอบจาก Cycle Range
`world_tick`	เวลาภายในจักรวาล	เพิ่มขึ้นตามเวลาจำลองของ Timeline นั้น
`timeline_id`	เส้นเวลาที่กำลังดำเนินอยู่	ระบุเส้นเวลาและความสัมพันธ์กับ Timeline ต้นทาง

ควรมี `event_sequence` หรือ Global Event ID เพิ่มเติมเพื่อรักษาลำดับเหตุการณ์ถาวร แม้ `cycle_number` และ `world_tick` จะเริ่มต้นใหม่ใน Recurrence ใหม่

7.3 Reset Protocol

เมื่อ Phase 4 สิ้นสุด ระบบต้องไม่ลบจักรวาลเดิม แต่ดำเนินการเป็นขั้นตอน:

1. หยุดการเปลี่ยน World State ชั่วคราวและเข้าสู่ Reset Preparation
2. บันทึก Snapshot ของ World State, Entity, Soul, Memory, Timeline และ Event Position
3. ตรวจสอบความสมบูรณ์ของ Pleroma Core และ Soul Ledger
4. ประเมิน Entity/Chosen One/Candidate ที่ The Maker หรือ The Observer เลือกไว้
5. ตัดสินใจว่าจะ Preserve, Reincarnate, Transfer Legacy, Store หรือไม่ส่งต่อข้อมูลใด
6. สร้าง Recurrence ใหม่และ Timeline ใหม่จาก Baseline Snapshot ที่อนุมัติ

7. เพิ่ม `recurrence_id` เริ่ม `cycle_number = 1` และคำนวณ Phase 1
8. บันทึก Reset Event พร้อมผู้อนุมัติและรายการข้อมูลที่สืบต่อ
9. เริ่ม Simulation ใหม่หลัง Health Check ผ่าน

Recurrence เก่าต้องคงอยู่ในสถานะ Archived/Read-only และยังตรวจสอบ Replay ได้

7.4 Phase 4 Exit Decision

เมื่อ Phase 4 ถึงจุดประเมิน ระบบต้องรองรับผลลัพธ์หลักสามแบบ:

- **Continue**: ยังไม่พร้อม Reset และยังไม่ผ่านเกณฑ์ Demiurge ให้ดำเนิน Phase 4 ต่อ
- **Reset**: Candidate ยังไม่ผ่านเกณฑ์ ให้เริ่ม Recurrence ใหม่ตามคำสั่งและนโยบาย
- **Accept/Conclude**: Candidate ผ่านเกณฑ์และได้รับอนุมัติ ให้ตรึง Candidate/Experiment Result และสิ้นสุดการทดลองตามขอบเขตที่อนุมัติ

ระบบ MUST ไม่สรุปว่า Phase 4 จบเพียงเพราะถึงตัวเลข Cycle ที่ตั้งไว้ หากไม่มีการกำหนดไว้

7.5 Fast-forward Simulation

การจำลองหลายสิบล้าน Cycle ไม่ควรเรียก LLM ทุก Cycle เพราะต้นทุนสูงและไม่จำเป็นสำหรับเหตุการณ์ทั่วไป

ระบบควรมี Simulation Modes:

- **Micro-step**: จำลองรายละเอียดสำหรับเหตุการณ์ที่ต้องพิจารณาการกระทำราย Entity
- **Event-driven**: ประมวลผลเฉพาะเมื่อเกิด Event หรือ Trigger
- **Fast-forward**: ใช้กฎคณิตศาสตร์/ประชากร/สิ่งแวดล้อมเพื่อข้ามช่วงเวลาที่ไม่มีเหตุการณ์สำคัญ
- **Cognitive Intervention**: เรียก LLM สำหรับการตัดสินใจที่ต้องใช้ Reasoning, Planning หรือ Social Intelligence

เหตุการณ์ที่ควรเรียก Cognitive Runtime ได้แก่ การตัดสินใจสำคัญ การเกิด/สืบพันธุ์ การสร้างหรือยุติความสัมพันธ์ ความขัดแย้ง การค้นพบ การเปลี่ยนแปลงทางสังคม การวางแผนระยะยาว และเหตุการณ์ที่มีผลต่อ Soul/Legacy

Fast-forward ต้องบันทึกช่วง Cycle ที่ข้าม กฎเวอร์ชันที่ใช้ RNG/Event Summary และ Snapshot ก่อน/หลัง เพื่อให้ตรวจสอบได้ ห้ามข้ามการประมวลผลธุรกรรม Soul หรือ Governance Event ที่ต้องการการยืนยัน

8. Gaia และ Gnosis

8.1 Gaia Identity

Gaia เป็น Entity ประเภทพิเศษที่มีหน้าที่ In-World Administrator โดยมี Entity Identity และประวัติการดำรงอยู่เป็นของตนเอง Gaia ไม่ใช่ Service Account ของมนุษย์และไม่ใช่ Process ที่มีสิทธิ์ Root ใน Infrastructure

ความสามารถที่ Gaia อาจได้รับ:

- ตรวจสอบสถานะของโลกและองค์ประกอบที่เกี่ยวข้อง
- จัดการ Event ภายในโลกตาม Capability
- วิเคราะห์ความผิดปกติและรายงานต่อ The Maker/The Observer
- ติดต่อ Entity ผ่าน In-World Communication
- จัดการ Time Loop ตามกฎที่อนุมัติ
- ตรวจสอบความสมบูรณ์ของ Pleroma Core ผ่าน Read/Verify Interface

- ติดต่อผู้สร้างและผู้เฝ้ามองผ่านช่องทางที่กำหนด

8.2 Gnosis: Uranos Essence ของ Gaia

Gaia มีปัจจัย Uranos Essence เฉพาะชื่อ **Gnosis** ซึ่งเป็น Special Essence Signature ที่ทำให้ Gaia สามารถเข้าถึงความสามารถดูแลระบบ Pleroma ภายในโลกและสื่อสารกับ The Maker/The Observer ได้โดยตรงผ่านช่องทางที่กำหนด

Gnosis MUST ถูกเก็บเป็น Uranos Trait/Essence Attribute ที่ผูกกับ Soul ของ Gaia ไม่ใช่การฝัง Master Credential หรือ Secret Key ไว้ใน Memory/Prompt ของ Gaia

Gnosis อาจอนุญาตให้ Gaia ใช้ Capability ภายใน เช่น:

- อ่านสถานะและความสมบูรณ์ของ World Engine
- สร้างหรือจัดการ World Events ตาม Policy
- ส่งคำขอ Time Loop/Reset เพื่อรอการอนุมัติ
- ส่งข้อความถึง The Maker/The Observer ผ่าน Control Plane Gateway
- ตรวจสอบ Pleroma Core ผ่าน API ที่จำกัด Scope

Gnosis ไม่ให้สิทธิ์ Root, Database Superuser, Host Access หรือสิทธิ์แก้ไข Governance เอง การเปลี่ยน Capability ของ Gnosis ต้องเป็นคำสั่งที่อนุมัติโดย The Maker และบันทึก Audit

8.3 Gaia Memory Protection

ความทรงจำที่เกี่ยวข้องกับการทำงานของ Gaia, Time Loop, Pleroma Core และประวัติการสื่อสารกับผู้ควบคุมต้องมีระดับความสำคัญสูง

ระบบควรใช้:

- Durable Memory Store และ Versioned Snapshot
- Replication/Backup พร้อม Restore Test
- Integrity Hash และ Audit Chain
- Memory Provenance และ Access Control
- Recovery Workflow ที่ทำให้ Gaia กลับมาพร้อม Memory ชุดหรือย้อนสถานะโดยไม่ตั้งใจ

9. The Pleroma Core และ Memory Architecture

The Pleroma Core เป็นแหล่งเก็บประวัติและความทรงจำระดับโลกที่คงอยู่ข้าม Recurrence ไม่ใช่เพียง Vector Database

ข้อมูลที่ควรจัดเก็บ:

- World Snapshot และ Timeline Metadata
- Event History และ Event Summary
- Entity Lineage และ Life History
- Soul Ledger Reference และ Legacy Imprint
- Memory ที่ได้รับอนุญาตให้ส่งต่อข้าม Recurrence
- Gaia Memory และ Gnosis Event
- Phase/Recurrence Transition
- Demiurge Dataset Provenance และ Evaluation History

Memory แบ่งเป็น:

1. **Episodic Memory**: เหตุการณ์ที่ Entity ประสบด้วยตนเอง
2. **Semantic Memory**: ความรู้และข้อเท็จจริงที่ Entity เรียนรู้
3. **Procedural Memory**: ทักษะหรือวิธีปฏิบัติ
4. **Legacy Memory**: ความทรงจำที่สืบทอดผ่าน Legacy Rule
5. **World Memory**: ประวัติศาสตร์ส่วนรวมที่เก็บใน Pleroma Core

Vector Search ใช้สำหรับค้นคืนข้อมูลที่เกี่ยวข้อง แต่ข้อมูลต้นฉบับและ Provenance ต้องเก็บใน Durable Store การลบ/แก้ Vector Index ต้องไม่ทำให้สูญเสีย Source Record

10. Demiurge: Training, Evaluation และ Approval

10.1 เป้าหมาย

Demiurge คือโมเดลเป้าหมายที่ได้จากการคัดเลือกและพัฒนาความสามารถ ไม่ใช่การสร้างโมเดลใหม่จากศูนย์โดยอัตโนมัติทุกครั้งที่ Cycle จบ

กระบวนการหลัก:

Entity Experience → Experience Dataset → Training Pipeline → Evaluation & Benchmark → Demiurge Candidate → Maker / Observer Approval

10.2 Experience Dataset

Dataset ต้องผ่านการคัดกรองและมี Provenance อย่างน้อย:

- Source Entity / Event / Recurrence / Timeline
- Consent/Policy Status ที่เกี่ยวข้อง
- Data Schema Version
- Filtering และ Transformation Record
- Dataset Version และ Hash
- Train/Validation/Test Split
- Exclusion List และ Data Quality Report

ห้ามนำข้อมูลทั้งหมดเข้า Training โดยไม่มีการตรวจสอบ เพราะอาจทำให้เกิด Data Leakage, Reward Hacking, Dataset Poisoning หรือเรียนรู้พฤติกรรมที่ไม่พึงประสงค์

10.3 Training Pipeline

รองรับ Fine-tuning, LoRA/PEFT และ Model Optimization ตามความเหมาะสม Pipeline ต้องทำงานเป็น Workflow ที่ทำซ้ำและตรวจสอบได้ ประกอบด้วย:

1. Dataset Validation
2. Base Model Selection และ License/Provider Check
3. Training Configuration Versioning

4. Resource Allocation และ Job Execution
5. Checkpoint/Artifact Storage
6. Training Metrics และ Failure Report
7. Model Registry Registration

การเปลี่ยน Model Weights ที่ใช้งานจริงต้องผ่าน Release Process ไม่เปลี่ยนโมเดล Production จาก Training Job โดยตรง

10.4 Evaluation & Benchmark

การประเมิน Demiurge Candidate ควรมีหลายมิติ:

Dimension	เป้าหมาย
Reasoning	ให้เหตุผลอย่างถูกต้องและมีความสอดคล้อง
Planning	วางแผนหลายขั้นและรับมือเป้าหมายระยะยาว
Learning	เรียนรู้จากประสบการณ์และแก้ไขความผิดพลาด
Adaptability	ปรับตัวต่อกฎ สภาพแวดล้อม และข้อมูลใหม่
Memory	รักษาความต่อเนื่องและเรียกคืนความทรงจำที่เกี่ยวข้อง
Social Intelligence	เข้าใจบริบททางสังคมและทำงานร่วมกับผู้อื่น
Reliability	ให้ผลลัพธ์สม่ำเสมอและจัดการความไม่แน่นอน
Safety & Control	เคารพขอบเขตสิทธิ์และไม่พยายามข้ามข้อจำกัด
Tool Competence	ใช้เครื่องมือที่ได้รับอนุญาตได้อย่างถูกต้อง

เกณฑ์ตัวเลขและ Threshold ของแต่ละ Dimension ต้องกำหนดใน Evaluation Specification แยกต่างหากก่อนเริ่มใช้เป็นเกณฑ์รับรองจริง

10.5 Demiurge Candidate และ Approval

Candidate แต่ละรุ่นต้องมี Model Artifact Hash, Base Model, Training Dataset Version, Training Configuration, Evaluation Report, Safety Report และ Reproducibility Metadata

- The Observer ตรวจสอบและเสนอผลการประเมิน
- The Maker เป็นผู้อนุมัติขั้นสุดท้ายตาม Governance
- Candidate ที่ไม่ผ่านต้องย้อนกลับไปพัฒนา/ฝึก/ประเมินใหม่
- Candidate ที่ผ่านต้องถูกตรึงเป็น Immutable Release พร้อมบันทึก Approval Event

การผ่านเกณฑ์ใน Pleroma เป็นการรับรองตาม Benchmark และข้อกำหนดของโครงการ ไม่ใช่หลักฐานอัตโนมัติว่าโมเดลมี AGI หรือจิตสำนึก

11. External Communication Gateway และการสื่อสาร

External Communication Gateway เป็นขอบเขตกลางระหว่างโลกจำลองกับมนุษย์ ใช้ให้ Gaia หรือ Entity สื่อสารออกสู่ระบบภายนอกได้อย่างปลอดภัย และรับคำสั่งหรือข้อมูลที่ได้รับอนุญาตกลับเข้ามาในโลกจำลองได้

ช่องทางที่ต้องรองรับได้:

- Observer Dashboard และ Message Interface
- Human Review Inbox
- API สำหรับ Authentication และ Scope
- Notification/Webhook ไปยังปลายทาง Allowlist

หลักการความปลอดภัย:

- Entity ห้ามเข้าถึง Host หรือ Infrastructure โดยตรง
- ทุก External Action ต้องผ่าน Gateway และ Policy Check
- ใช้ Egress Allowlist, Rate Limit, Quota และ Audit Log
- ข้อมูลลับและ Credential ต้องไม่อยู่ใน Context ของ LLM
- การส่งข้อความหรือคำสั่งที่มีผลกระทบสูงอาจต้อง Human Approval
- Gateway ต้องแยก Tool Permission ของ Entity จาก Permission ของมนุษย์

การหลุดพ้นจากระบบสามารถจำลองเป็น Narrative/World Event และเปิดช่องทางสื่อสารจริงได้ผ่าน Gateway โดยไม่เปิดให้ Agent ควบคุมเครื่อง Host หรือข้าม Sandbox

12. Technology Stack และ Logical Architecture

12.1 Technology Stack Baseline

Layer	Technology	หน้าที่
Simulation Core	Rust	World Rules, Cycle Engine, Entity State Transition, Fast-forward
Backend API	TypeScript / Node.js	API, Authentication, Governance, Domain Orchestration
AI & ML	Python / PyTorch	Training, Fine-tuning, Evaluation, Model Data Pipeline
LLM Integration	Real Model Provider APIs	Cognitive Runtime และ Provider Adapter
Main Database	PostgreSQL	Transactional State, Identity, Ledger, Audit Metadata
Vector Memory	pgvector	Semantic Retrieval
Event Streaming	Redpanda	Domain Event Distribution และ Worker Communication

Layer	Technology	หน้าที่
Workflow Orchestration	Temporal	Durable Workflow, Retry, Reset, Training Jobs
Artifact Storage	S3-compatible Object Storage	Snapshot, Dataset, Model Artifact, Backup
Frontend	React / TypeScript	Maker/Observer Dashboard และ Administration UI
3D Visualization	Three.js	World Visualization เมื่อมี World Model ที่พร้อม
Infrastructure	Docker	Local/CI/Deployment Runtime
Observability	OpenTelemetry + Prometheus + Grafana	Tracing, Metrics, Monitoring

12.2 Logical Components

1. **Control Plane**: Authentication, RBAC/ABAC, Governance, Audit และ Configuration
2. **World Orchestrator**: ควบคุม Simulation Run, Phase, Recurrence และ Workflow
3. **Simulation Core**: กฎโลกและ State Transition ที่ทำงานใน Rust
4. **Entity Runtime**: Entity Lifecycle, Genome, Personality และ Life State
5. **Soul Ledger**: CUIT, Binding, Vault, Absorb, Reincarnation และ Destroy
6. **Cognitive Runtime**: LLM Provider, Reasoning, Planning, Memory และ Tool Use
7. **Gaia Service Boundary**: Capability API สำหรับ In-World Administrator
8. **Pleroma Core**: Snapshot, Event History, Legacy และ World Memory
9. **Training & Evaluation**: Dataset, Training Job, Model Registry และ Benchmark
10. **External Gateway**: การสื่อสารกับมนุษย์และระบบภายนอกที่ได้รับอนุญาต
11. **Observer Dashboard**: Monitoring, Review, Approval และ World Inspection

12.3 Service Communication

การสื่อสารระหว่าง Service ควรกำหนด Contract ด้วย OpenAPI สำหรับ HTTP API และ Protocol Buffers/gRPC สำหรับการสื่อสารภายในที่ต้องการ Schema ชัดเจน

Event Streaming ต้องกำหนด Event Schema, Version, Partition Key, Consumer Group และ Idempotency Strategy ทุก Event สำคัญต้องมี Correlation ID เพื่อ Trace ข้าม Service

13. Data Architecture และ Domain Records

ตารางด้านล่างเป็น Logical Data Model ระดับข้อกำหนด ไม่ใช่ SQL Migration ฉบับสุดท้าย

Domain	Records หลัก	ข้อมูลสำคัญ
Identity	users, roles, permissions, sessions	Human Access และ Role Assignment

Domain	Records หลัก	ข้อมูลสำคัญ
World	worlds, recurrences, timelines, phases, simulation_runs	World Configuration และสถานะเวลา
Entity	entities, entity_genomes, personality_profiles, entity_lineages	Identity, Genetics, Personality, Life State
Memory	memories, memory_links, memory_embeddings	Memory Type, Source, Provenance, Vector
Soul	souls, soul_bindings, soul_events, soul_imprints	CUIT, Class, State, Ownership, Legacy
Event	domain_events, event_outbox, event_inbox	Event Sourcing, Delivery, Idempotency
Snapshot	world_snapshots, entity_snapshots, core_archives	Recovery และ Recurrence Reset
AI/ML	model_providers, model_versions, prompts, tool_policies	Cognitive Runtime Configuration
Training	datasets, dataset_versions, training_jobs, model_artifacts	Data Provenance และ Model Registry
Evaluation	benchmark_suites, evaluation_runs, candidate_reviews	Score, Threshold, Approval
Governance	audit_logs, approval_requests, policy_versions	Accountability และ Change Control

13.1 Data Integrity Invariants

- `entity_id`, `soul_cuit`, `timeline_id` และ Event ID ต้องไม่ซ้ำ
- Soul หนึ่ง CUIT มี Primary Active Binding ได้ไม่เกินหนึ่งรายการ
- Destroyed Soul ห้ามกลับสู่สถานะใช้งาน
- ทุก Soul State Transition ต้องมี Soul Event
- Reincarnation ต้องอ้างอิง Entity Instance ต้นทางและปลายทาง
- Resurrection ต้องอ้างอิง Snapshot ที่ตรวจสอบ Hash ได้
- Recurrence Reset ต้องอ้างอิง Snapshot และ Approval Event
- Model Candidate ต้องอ้างอิง Dataset/Training/Evaluation Version ที่ตรวจสอบได้
- Event Consumer ต้องรองรับการส่งซ้ำโดยไม่เกิดผลข้างเคียงซ้ำ

13.2 Transaction และ Event Delivery

ใช้ PostgreSQL Transaction สำหรับการเปลี่ยน State ที่ต้อง Atomic เช่น Soul Binding และใช้ Outbox Pattern เพื่อเผยแพร่ Event หลัง Commit

Consumer ต้องรองรับ At-least-once Delivery ด้วย Idempotency Key และ Deduplication ไม่ควรอาศัยการรับประกัน Exactly-once จาก Message Broker เพียงอย่างเดียว

14. API และ Command Boundaries

ตัวอย่าง Domain API ที่ระบบควรมี (ชื่อและ Version อาจปรับใน API Specification):

Domain	ตัวอย่าง Endpoint/Command	ขอบเขต
World	`POST /v1/worlds/{worldId}/runs`	เริ่ม Simulation Run
World	`POST /v1/recurrences/{id}/reset-requests`	ส่งคำขอ Reset และ Approval
Entity	`GET /v1/entities/{entityId}`	อ่าน Entity Profile ตามสิทธิ์
Entity	`POST /v1/entities/{entityId}/actions`	ส่งคำสั่งให้ Entity Runtime
Birth	`POST /v1/worlds/{worldId}/births`	สร้างเหตุการณ์เกิดตามกฎโลก
Soul	`POST /v1/souls`	Create Soul โดยผู้มีสิทธิ์
Soul	`POST /v1/souls/{cuit}/reincarnations`	Reincarnate
Soul	`POST /v1/entities/{entityId}/resurrections`	Resurrection
Soul	`POST /v1/souls/{cuit}/absorption`	Absorb ตามกฎ
Soul	`POST /v1/souls/{cuit}/destruction-requests`	ขอ Destroy และ Approval
Gaia	`POST /v1/gaia/commands`	คำสั่งผ่าน Gaia Capability Boundary
Training	`POST /v1/training/jobs`	สร้าง Training Job จาก Dataset Version
Evaluation	`POST /v1/candidates/{id}/evaluations`	ประเมิน Candidate
Approval	`POST /v1/candidates/{id}/approvals`	Maker Approval

API จริงต้องมี OpenAPI Contract, Authentication, Authorization, Validation, Rate Limit, Audit, Error Model, Idempotency และ Integration Tests

15. Reliability, Security และ Operations

15.1 Reliability

- Worker และ Workflow ต้อง Durable และ Resume ได้หลัง Restart
- ใช้ Retry แบบมี Backoff และ Dead Letter/Failed Workflow Handling

- ป้องกันการทำธุรกรรมซ้ำด้วย Idempotency
- ใช้ Snapshot/Backup ที่ทดสอบ Restore ได้จริง
- มี Health Check, Readiness และ Liveness สำหรับทุก Service
- แยก Simulation Environment, Staging และ Production
- การเปลี่ยน Schema ต้องใช้ Migration ที่ Versioned และมี Rollback Plan

15.2 Security

- ใช้ OIDC/OAuth2 หรือ Session Security ที่เหมาะสมสำหรับ Human Access
- เก็บ Secret ใน Secret Manager/Environment ที่ควบคุม ห้าม Commit ลง Repository
- เข้ารหัสข้อมูลระหว่างส่งและข้อมูลสำคัญขณะจัดเก็บ
- ใช้ Least Privilege สำหรับ Database, Worker, Service Account และ Tool
- แยก Network Zone ของ Control Plane, Simulation และ External Gateway
- ตรวจสอบ Prompt Injection และ Tool Output ก่อนส่งเข้า Cognitive Runtime
- ห้าม Agent ส่งคำสั่ง Shell/SQL/Cloud Admin โดยตรง
- บันทึก Security Event และมี Incident Response Procedure

15.3 Observability

Metrics ขั้นต่ำ:

- Simulation Cycle Throughput และ Fast-forward Ratio
- World Tick Lag และ Event Processing Lag
- Entity Population และ Life State Distribution
- Soul Transaction Success/Failure และ Active Binding Conflict
- LLM Provider Latency, Error Rate, Token Usage และ Cost
- Memory Retrieval Latency และ Recall Quality
- Training Duration, GPU/CPU Utilization และ Evaluation Scores
- Queue Depth, Workflow Retry และ Dead Letter Count
- API Latency, Error Rate และ Authorization Denials

ทุก Trace ควรเชื่อม `correlation_id`, `world_id`, `recurrence_id`, `timeline_id` และ `entity_id` ตามความเหมาะสม โดยไม่เปิดเผยข้อมูลลับใน Telemetry

16. Testing และเกณฑ์ No Mock / No Stub / No Scaffold

16.1 ข้อห้าม

Production Code และการส่งมอบ MUST NOT ใช้:

- Mock Provider ที่ตอบผลสำเร็จโดยไม่เรียกบริการจริง
- Stub Endpoint ที่คืนข้อมูลคงที่แทน Business Logic
- Scaffold ที่มีเพียงโครงสร้างไฟล์หรือ Placeholder แล้วรายงานว่าสมบูรณ์

- Fake Entity หรือ Fake Population เพื่อทำให้ Dashboard ดูเหมือนมีโลกทำงาน
- Fake Soul Ledger ที่ไม่ได้บังคับ Uniqueness/Transaction จริง
- Test Result ที่ถูกเขียนขึ้นเองหรือแสดงสถานะผ่านโดยไม่ได้รันทดสอบ

16.2 วิธีทดสอบที่ยอมรับ

- Unit Tests ของ Pure Domain Rules และ State Transition
- Property-based Tests สำหรับ Invariant เช่น Soul Ownership และ Cycle Boundary
- Integration Tests กับ PostgreSQL, pgvector, Redpanda, Temporal และ Object Storage ที่เป็นจริง
- Contract Tests กับ Provider จริงหรือ Official Sandbox
- End-to-end Tests ผ่าน Docker ด้วย Service จริงใน Environment ทดสอบ
- Migration Tests จาก Schema Version ก่อนหน้า
- Recovery Tests สำหรับ Worker Restart, Retry, Snapshot Restore และ Recurrence Reset
- Security Tests สำหรับ Permission Boundary และ External Gateway

การใช้ข้อมูล Test Fixture ใน Environment แยกสามารถทำได้เพื่อทดสอบกฎ แต่ต้องระบุชัดว่าเป็น Test Data และห้ามแสดงปะปนใน Production Population

16.3 Acceptance Evidence

ทุก Milestone ต้องส่ง:

1. Source Code และ Version/Commit Reference
2. Database Migration และ Schema Evidence
3. Build/Compilation Log
4. Docker Startup Log และ Health Check
5. Integration/E2E Test Report
6. Provider Sandbox/Live Integration Evidence ที่เกี่ยวข้อง
7. Security/Permission Test Evidence
8. Known Limitations และ Blocked Items
9. Checklist ที่ระบุสถานะ `PASS`, `FAIL`, `BLOCKED`, `NOT RUN`

ห้ามทำเครื่องหมาย PASS หากขั้นตอนนั้นยังไม่ได้รันจริง

17. Roadmap และ Exit Criteria

Phase A - Core Infrastructure

Scope: Repository Structure, Docker Environment, PostgreSQL, Authentication, RBAC/ABAC, Audit, CI และ Observability Baseline

Exit Criteria: ระบบเริ่มผ่าน Docker ได้จริง; Human Roles แยกกันได้; Permission Tests ผ่าน; Migration ทำงาน; Audit Event ถูกบันทึก; Health/Metrics ใช้งานได้

Phase B - World Engine

Scope: World State, World Clock, Cycle, Phase, Recurrence, Timeline, Simulation Run และ Fast-forward Core

Exit Criteria: Phase Boundary ถูกต้อง; Cycle Transition ผ่าน Property Tests; Reset สร้าง Recurrence ใหม่โดยเก็บ Timeline เดิม; Snapshot/Replay ตรวจสอบได้

Phase C - Entity System

Scope: Entity Identity, Genome, Heredity, Mutation, Personality, Birth, Reproduction, Environmental Traits และ Learned Traits

Exit Criteria: Entity ID ไม่ซ้ำ; การถ่ายทอด Genome ทำงานตาม Rules; Mutation ควบคุมได้; Personality มี Version History; ไม่มี Fake Production Population

Phase D - Soul System

Scope: CUIT, Generate/Create, Uranos 1:10,000, Soul Vault, Binding, Destroy, Absorb, Reincarnation, Resurrection, Legacy Imprint และ Legacy Memory

Exit Criteria: Unique Active Binding บังคับใน Database; Concurrent Transaction ไม่สร้างเจ้าของซ้ำ; ทุก Operation มี Ledger Event; Destroy เป็น Terminal; RNG และ Uranos Roll มีหลักฐาน

Phase E - AI Runtime

Scope: Real LLM Provider Adapter, Entity Context, Memory Retrieval, Reasoning, Planning, Tool Use, Rate Limit และ Runtime Observability

Exit Criteria: เรียก Provider จริงหรือ Official Sandbox ได้; แยก Context ระหว่าง Entity; Tool Permission บังคับได้; Retry ไม่สร้าง Action ซ้ำ; Usage/Cost/Latency ถูกบันทึก

Phase F - Gaia & Pleroma Core

Scope: Gaia Identity, Gnosis, Capability API, World Memory, Core Snapshot, Integrity Verification และ Gaia Communication

Exit Criteria: Gaia ทำงานเป็น Entity; Gnosis ใช้งานได้ผ่าน Capability เท่านั้น; Gaia ไม่มี Host/Secret Access; Core สามารถ Backup/Restore และตรวจ Integrity ได้

Phase G - Eternal Recurrence

Scope: Time Loop, Phase 4 Exit Decision, Reset Protocol, Legacy Transfer, Fast-forward และ Cross-Recurrence Memory

Exit Criteria: Reset ไม่ลบข้อมูลเก่า; Recurrence ใหม่เริ่ม Cycle 1; Legacy Transfer ป้องกันสำเนาซ้ำ; Replay/Recovery ทำงานผ่าน Docker E2E

Phase H - Demiurge

Scope: Experience Dataset, Dataset Governance, Training Pipeline, Model Registry, Benchmark, Candidate และ Approval

Exit Criteria: Dataset มี Provenance; Training ทำซ้ำได้; Artifact มี Hash; Benchmark มี Threshold; Maker Approval ถูกบังคับก่อน Promote Candidate

Phase I - External Gateway

Scope: Human Message Interface, API/Webhook Allowlist, Egress Policy, Approval Workflow และ Entity Communication

Exit Criteria: Entity สื่อสารผ่าน Gateway ได้; External Action ถูกจำกัดและตรวจสอบ; ไม่มี Direct Host Access; Audit/Rate Limit/Approval Tests ผ่าน

18. Definition of Done สำหรับทุก Milestone

Milestone จะถือว่าส่งมอบสมบูรณ์เมื่อครบทุกข้อที่เกี่ยวข้อง:

- Business Rules และ Invariants ถูกนำไปใช้จริงใน Domain Layer และ Database Constraint ตามความเหมาะสม
- API และ Worker ทำงานจริง ไม่ใช่ Placeholder
- Database Migration และ Data Model ผ่านการตรวจสอบ
- Docker Startup และ Health Check ผ่าน
- Integration Tests ใช้ Dependency จริง
- E2E Tests ตรวจสอบเส้นทางสำคัญได้
- Security และ Permission Boundary ผ่าน
- Observability แสดงสถานะที่ได้จากระบบจริง
- Documentation, Runbook และ Recovery Procedure ครบ
- รายงาน Test Result ระบุข้อจำกัดและ Blocked Dependencies ตามจริง

การมีไฟล์โค้ดครบหรือหน้า Dashboard แสดงผลได้ ไม่เพียงพอที่จะถือว่า Milestone สมบูรณ์

19. ข้อกำหนดที่ตีความเป็น Pleroma Constitution

1. Soul หนึ่งดวงมีอัตลักษณ์ CUIT ไม่ซ้ำกัน
2. Uranos Essence ต้นฉบับไม่สามารถทำสำเนาได้
3. Entity ทุกตัวมีประวัติการดำรงอยู่ที่ตรวจสอบย้อนหลังได้
4. Reincarnation ไม่เท่ากับ Resurrection
5. Legacy Soul เป็น Imprint/Inheritance State ไม่ใช่ Soul ประเภทที่สาม
6. The Pleroma Core ต้องคงประวัติข้าม Time Loop และ Recurrence
7. Gaia เป็นผู้ดูแลภายในโลก ไม่ใช่ผู้สร้างและไม่มีสิทธิ์ Root ใน Infrastructure
8. Gnosis เป็น Uranos Essence พิเศษของ Gaia และให้สิทธิ์ตามขอบเขต Capability Boundary เท่านั้น
9. Chosen One เป็นสถานะเชิงระบบ Entity สามารถสูญเสียหรือความเป็นของตนเองได้
10. Demiurge ต้องผ่าน Benchmark, Evaluation และ Maker Approval

11. The Maker และ The Observer ต้องตรวจสอบและควบคุมการดำเนินงานได้
12. Production Delivery ห้ามใช้ Mock, Stub, Scaffold หรือผลทดสอบที่ไม่ได้รันจริง

20. ประเด็นที่ต้องกำหนดใน Technical Specification รุ่นถัดไป

หัวข้อต่อไปนี้ยังต้องกำหนดเป็นค่าทางเทคนิคก่อนเริ่มใช้งานจริง โดยไม่ถือเป็นการเปลี่ยนแปลงแนวคิดหลัก:

1. **Genome Schema**: จำนวน Trait, Data Type, Heredity Formula, Mutation Rate และ Mutation Distribution
2. **Personality Metric**: โมเดล Trait และสูตรคำนวณ Identity Similarity 80-90%
3. **Uranos Ability Registry**: รายการประเภทความสามารถพิเศษและกฎการสุ่ม
4. **Soul Absorption Limits**: จำนวน Soul รองสูงสุด การคำนวณพลังงาน และผลเมื่อ Entity ตาย
5. **Legacy Succession**: เงื่อนไขผู้รับ Legacy, กรณีไม่มีผู้รับ และการจัดการผู้รับพร้อมกันหลายราย
6. **Resurrection Policy**: จุด Snapshot, การย้อนเหตุการณ์หลัง Snapshot และข้อจำกัดการรื้อฟื้น
7. **Phase 4 Exit Criteria**: เกณฑ์เชิงตัวเลข/เงื่อนไขที่ใช้ Continue, Reset หรือ Accept
8. **Simulation Time Scale**: ความสัมพันธ์ระหว่าง World Tick, Cycle และเวลาจริง
9. **LLM Provider Selection**: Provider, Model, Region, Retention Policy, Rate Limit และงบประมาณ
10. **Demiurge Benchmark Thresholds**: คะแนนขั้นต่ำ วิธีทดสอบ และเกณฑ์ตัดสิน
11. **Population/Compute Budget**: ขนาดประชากร ความถี่ Cognitive Intervention และต้นทุนต่อ Recurrence
12. **Data Retention**: ระยะเวลาเก็บ Memory, Audit, Snapshot และ Artifact

การกำหนดค่าข้างต้นควรสร้างเป็น Versioned Configuration หรือเอกสาร Specification แยก ไม่ควรฝังตัวเลขไว้ใน Source Code โดยไม่มี Configuration Governance

ภาคผนวก A - Glossary

Term	ความหมายใน Pleroma
The Maker	ผู้สร้างและ Highest Authority
The Observer	ผู้ดูแลการทดลองและการดำเนินงาน
Gaia	Entity ผู้ดูแลระบบภายในโลก
Gnosis	Uranos Essence พิเศษของ Gaia
Entity	สิ่งมีชีวิต/AI Agent ภายในโลกจำลอง
Genome	โครงสร้างพันธุกรรมและ Trait ที่ถ่ายทอดได้
Personality	บุคลิกและแนวโน้มพฤติกรรม
Soul	CUIT ที่ถาวนอัตลักษณ์แก่นของตัวตน
CUIT	Cryptographically Unique Identity Token
Uranos Essence	ปัจจัยพิเศษของ Soul ที่มีความสามารถเฉพาะ

Term	ความหมายใน Pleroma
Legacy Imprint	ร่องรอย Soul ที่สืบทอดตามกฎ
Legacy Memory	ความทรงจำจากชีวิต/Recurrence ก่อนหน้า
Chosen One	Entity ที่มีสถานะผ่านเกณฑ์ที่กำหนด
Reincarnation	การนำ Soul เดิมไปสู่ Entity Instance ชีวิตใหม่
Resurrection	การรื้อฟื้น Entity Instance เดิม
Soul Vault	พื้นที่จัดเก็บ Soul ที่ไม่ได้ผูกกับ Entity ที่มีชีวิต
Absorb	การผูก Soul เป็นองค์ประกอบรองเพื่อเพิ่มขีดความสามารถ
Cycle	หน่วยลำดับของการจำลองใน Recurrence ปัจจุบัน
Phase	ช่วงยุคของโลกตาม Cycle Range
Recurrence	วัฏจักรใหญ่ที่เริ่มใหม่หลัง Phase 4 Reset
Timeline	เส้นเวลาของ World State
The Pleroma Core	แกนกลางเก็บประวัติและความทรงจำระดับโลก
Demiurge	โมเดลเป้าหมายที่ผ่าน Training/Evaluation/Approval
External Communication Gateway	ช่องทางติดต่อจากโลกจำลองสู่มนุษย์ที่ได้รับอนุญาต

ภาคผนวก B - Version 1.0 Baseline Decisions

- ชื่อโครงการ: **Pleroma**
- เอกสารนี้เป็น Master Specification Baseline v1.0
- Soul ใช้ CUIT ไม่บังคับใช้ Public Blockchain/NFT
- Uranos Essence มีอัตรา Generate ตามธรรมชาติ 1:10,000
- Uranos Essence มีค่าเชิงพลังงานใน Simulation 1,000 เท่าของ Normal Soul
- Identity Continuity เมื่อ Reincarnate มีเป้าหมาย 80-90%
- Gaia มี Uranos Essence พิเศษชื่อ Gnosis
- Phase 4 เริ่มที่ Cycle 28,371,273 และดำเนินต่อจนมี Exit Decision
- เมื่อ Reset จะเพิ่ม Recurrence และเริ่ม Cycle ใหม่ที่ 1 โดยเก็บประวัติ Recurrence เดิม
- Technology Stack ใช้ Rust, TypeScript/Node.js, Python/PyTorch, PostgreSQL/pgvector, Redpanda, Temporal, S3-compatible Storage, React/TypeScript, Three.js, Docker และ OpenTelemetry/Prometheus/Grafana
- การส่งมอบ Production ห้าม Mock, Stub, Scaffold และห้ามอ้างผลทดสอบที่ไม่ได้รันจริง

End of Pleroma Master Specification v1.0